HYBRID NETWORK TRAFFIC DETECTION SYSTEM

Fourth-year capstone | Technical proposal and implementation roadmap

Prepared 14 October 2026 | Planning report based on the uploaded project description and our project discussion. This report is a proposal, not a completed-system evaluation.

1. Executive summary

Develop a practical hybrid Network Intrusion Detection System (NIDS) that combines machine-learning classification with transparent rule-based network-flow analysis. Target an operational prototype that can ingest recorded PCAP traffic and, when authorized, live traffic; produce explainable alerts; and display results in a desktop-accessible local web dashboard.

Recommended deliverable: Python CLI detection engine + FastAPI service + React dashboard, with SQLite alert history. A mobile application and packaged desktop installer are optional enhancements, not core requirements.

2. Original project brief and objective

The uploaded brief identifies DoS/DDoS, port scans and brute-force attacks as target threats. It proposes combining pre-trained ML classifiers with flow-based rules using the CICIDS2017 dataset, with a goal of at least 95% detection accuracy and fewer false positives. Its motivation is an affordable intrusion-detection solution for educational institutions.

Important interpretation: The 95% target is an aspiration stated in the project brief, not a measured outcome. The implementation plan recommends training and documenting your own model and validating performance beyond overall accuracy.

3. Product format and practical use

- **Primary:** localhost web dashboard for a security analyst, teacher or lab demonstrator.
- **Engine:** standalone CLI to process supported CSV/PCAP inputs, emit JSON alerts and support automation.
- **Live mode:** authorized packet capture on a test network or machine, with explicit permissions and interface selection.
- **Users:** small security labs, student SOC demonstrations, and educational network monitoring; do not claim enterprise-grade protection without operational validation.

4. High-level system architecture

- **1. Input:** Authorized live packets or uploaded PCAP; labeled CICIDS2017 CSV files for training and evaluation.
- **2. Flow extraction:** Convert packet streams to bidirectional flows and timing, length, protocol and volume features.
- **3. Feature parity:** Apply the same schema, preprocessing and column order to training and inference.
- **4. Parallel detection:** ML model scores each supported flow while rules detect suspicious rates, destinations and connection patterns.
- **5. Fusion and persistence:** Combine predictions and rule evidence, label severity, and store events in SQLite.
- **6. Presentation:** FastAPI endpoints serve live alerts, charts, searchable history, case details and exports.

5. Scope and caution

Threat family	ML task	Rule-based evidence
DoS / DDoS	Identify suspicious high-volume flow characteristics	Traffic-rate spikes, concurrent connections or bursts
Port scanning	Recognize scan-like flow patterns	One source contacting many destination ports/hosts
Brute-force suspicion	Identify anomalous connection patterns	Authentication-failure events only if relevant system logs are integrated

Encrypted network flows do not generally reveal whether a login attempt succeeded or failed. Separate anomalies, suspected attacks and corroborated evidence. Do not equate an ML score with proof of an attack.

6. Recommended implementation stack

Layer	Suggested tools
Python and analysis	Python 3.12; pandas, NumPy, scikit-learn
GPU-accelerated tabular model	XGBoost with device="cuda"; Random Forest CPU baseline
Traffic handling	Scapy or PyShark; carefully validated CICFlowMeter-compatible flow extractor
API and UI	FastAPI; React + Tailwind and charting library
Persistence and quality	SQLite; pytest; Git/GitHub; experiment logs and model artifacts

7. Model development and evaluation

- Begin with CICIDS2017 labeled flows; document labels, missing values, duplicate records, class imbalance and leakage risks.
- Establish a fixed feature contract before attempting live inference. Validate feature parity using sample PCAPs.
- Start with binary benign-versus-malicious classification, then add attack-family categories if justified.
- Compare Logistic Regression or a simple baseline, Random Forest and XGBoost; tune using training/validation data only.
- Use scenario-separated or time-aware held-out tests where feasible to reduce leakage from related flows.
- Report precision, recall, F1, PR-AUC, false-positive rate, per-class confusion matrices, latency and throughput.
- Evaluate rule-only, ML-only and hybrid variants under the same protocol. Do not assume hybrid fusion improves metrics until demonstrated.

8. RTX 4050 usage

The laptop has an NVIDIA GeForce RTX 4050 Laptop GPU with about 6 GB VRAM and a CUDA-capable driver. CUDA-enabled PyTorch has already been verified in a separate synthetic neural-network experiment. The measured 0.73 s GPU vs 7.51 s CPU comparison (about 10.3x on that workload) confirms GPU computation is functional, but **does not predict speedup for CICIDS2017 or XGBoost**.

For this project, train Random Forest on the CPU and compare against XGBoost histogram training on the GPU. Use float32 data, manageable feature sets, memory monitoring and CPU fallback; 6 GB VRAM limits GPU-resident training sizes.

9. Ten-week capstone plan

Schedule	Deliverable
Week 1	Requirements, threat scope, repository, data audit and feature schema
Weeks 2-3	Clean data, train baseline models, GPU XGBoost experiment and save metrics
Weeks 4-5	Rules, hybrid decision policy and PCAP replay CLI with tests
Weeks 6-7	Feature-parity checks, authorized capture, FastAPI and persistence
Week 8	Dashboard, investigation views, filtering and export
Weeks 9-10	Ablation studies, performance testing, report, presentation and demo

Minimum viable milestone

A command-line program that ingests a documented supported input, emits reproducible alerts with evidence, and passes automated tests by the end of Week 5. This protects the capstone if user-interface work takes longer.

10. Team responsibilities and grading evidence

- Primary developer: architecture, model training and detection integration.
- Second member: dataset audit, references and reproducible experiment metadata.
- Third member: dashboard usability tests, test cases and demo capture.
- Fourth member: documentation, slides, testing coordination and report formatting.

Collect evidence for technical implementation, working demonstrations, properly controlled experiments, documented contributions, diagrams, reproducibility and clear viva explanations. The university rubric was not provided; any weighting from prior discussion is only illustrative.

11. Demonstration and acceptance checklist

- Demonstrate normal flow processing, then replay an authorized labeled PCAP or test trace.
- Show at least one alert with source/destination, time range, rule evidence and ML score.
- Compare alerts produced by rules only, ML only and hybrid fusion.
- Export alert records and present measured false positives, performance and known limitations.
- Record the exact dataset split, code revision, model version and hardware used for reported numbers.

12. Risks, ethics and resume relevance

Major risks: train/serve feature mismatch; benchmark leakage; overfitting to a historical dataset; false positives; inability to infer encrypted authentication outcomes; scope creep in frontend implementation.

Safe deployment: capture only on systems and networks you own or are explicitly authorized to inspect. Prefer local labs and recorded PCAPs for attack demonstrations.

Resume value: strongest when the finished project proves end-to-end engineering, explainable detection, repeatable evaluation and measured performance. Avoid describing it as production-ready or claiming accuracy without supporting held-out results.

13. Immediate next action

Create the repository, define the flow feature schema and dataset splitting policy, inspect a small CICIDS2017 sample, and train one CPU baseline before building the UI. Then establish PCAP-to-flow compatibility before attempting any real-time performance claim.

Basis of this report: Uploaded “Hybrid Network Traffic Detection System” project brief and the previously prepared “Hybrid NIDS — Chat & Capstone Plan.” The project architecture, technology decisions and schedule are recommendations, not original-brief requirements.